A Framework for Content-Keyed CRDT Convergence

Author: Subrata Sadhu
Affiliation: Independent Researcher
Contact: sadhu.arka507@gmail.com **Date:** 2026-07-18
Status: Draft v2 — Working Paper

Abstract

We identify a previously unnamed pattern in distributed systems: *content-keyed CRDTs* (CK-CRDTs) — a CRDT subclass whose merge partitions operations by a content-derived key and selects one representative per class. This pattern appears in content-addressed storage (IPFS), version control (Git), deduplicating sync, and entity resolution, but has never been formalized as a CRDT subclass. We prove four main results: (1) representative-selection via argmax is monotone and content-stable (Theorem 1); (2) canonicalization-at-write-time suffices for no-orphan guarantees under downstream CRDTs with foreign-key dependencies (Theorem 2); (3) the information loss of CK-CRDT merge is exactly the within-class loser set, and this is a tight lower bound for any merge satisfying our content-key properties (Lemmas 1–2); (4) three content-key properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each property is individually required (Theorem 4). We also show that composite keys inherit convergence, adaptive keys converge under acyclic migration, and CK-CRDTs compose with delta-CRDTs under stratified delta computation (§9). The framework classifies Docker, IPFS, Git, Yjs, Automerge, and Loro as instances or non-instances, explaining when content-keying is required and when ID-at-creation suffices.

---

1. Introduction

1.1 The Content-Keying Problem

Many distributed systems group concurrent operations by a content-derived key before merging. Examples:

- **Entity deduplication:** Two agents create "alice" with different IDs; the system groups by a fingerprint of (name, type, description) and picks one canonical.
- **Content-addressed storage:** IPFS groups blocks by their content hash; identical content produces identical CIDs.
- **Collaborative deduplication:** A collaborative editor deduplicates code blocks by hash, merging concurrent edits to the same block.
- **Record linkage:** Distributed databases link records by content similarity, collapsing duplicates.

These systems share a pattern: the merge function partitions operations by a content-derived equivalence relation, then selects one representative per class. We call this pattern *content-keyed CRDT* (CK-CRDT).

1.2 Why a Framework Is Needed

To our knowledge, no existing CRDT work explicitly formalizes the content-keyed partitioning pattern — previous systems implement it ad hoc. Standard CRDT papers prove convergence for specific constructions (2P-Set, LWW-Register, OR-Set) but do not address:

- What properties must the content key have for convergence? (Theorem 4: K1–K3 are necessary and sufficient under argmax selection. A counterexample shows each property is individually required.)
- How does the key interact with LWW or causal ordering? (Theorem 1: monotonicity and content-stability are equivalent for argmax selection.)
- What information is lost by content-keyed merge? (Lemmas 1–2: exactly the within-class loser set, and this is a tight lower bound.)
- When does content-keying compose correctly with downstream CRDTs? (Theorem 2: canonicalization-at-write-time is sufficient.)

The necessity direction — proving that convergence *requires* K1–K3 under argmax selection, not just that K1–K3 *imply* convergence — is the strongest result. Most CRDT frameworks prove sufficiency; we prove both directions.

1.3 Scope and Companion

This paper is the theoretical companion to Sadhu [14], which provides the reference implementation (`crdt_projection.py`, 51 passing tests) and empirical evaluation of a specific CK-CRDT instance. Paper 2 proves the general properties that Paper 1's pipeline satisfies. The two papers are complementary: Paper 1 is engineering + specific results; Paper 2 is theory + general framework.

1.4 Contributions

The paper makes four main contributions:

1. **Content-Key Monotonicity** (Theorem 1, §3): for argmax representative-selection, monotonicity and content-stability are equivalent. This establishes the structural foundation for CK-CRDT merge.

2. **Layered No-Orphan Invariant** (Theorem 2, §4): for any composition of a CK-CRDT followed by a downstream CRDT with foreign-key dependencies, the no-orphan invariant holds if the downstream CRDT applies canonicalization at write time.

3. **Information-Loss Characterization** (Lemmas 1–2, §5): CK-CRDT merge discards exactly the within-class loser set (Lemma 1), and this is a tight lower bound — no K1-K3-compliant merge can discard fewer operations (Lemma 2).

4. **Content Key Properties** (Theorem 4, §6): three properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each property is individually required.

We additionally show (§9) that composite keys inherit convergence, adaptive keys converge under acyclic migration, and CK-CRDTs compose with delta-CRDTs under stratified delta computation. The framework classifies Docker, IPFS, Git, Yjs, Automerge, and Loro as instances or non-instances (§7).

1.5 Related Work

1.5.1 CRDT Foundations Shapiro et al. [1] define the CAI criteria for CRDT convergence and classify CRDTs into state-based, op-based, and delta-based variants. Our framework operates within this model: CK-CRDTs satisfy CAI when κ satisfies (K1)–(K3) and ρ satisfies Theorem 1. Preguiça et al. [10] address fault-tolerant geo-replication with causal consistency, demonstrating that CRDT-based systems can maintain consistency across wide-area networks — a setting where CK-CRDTs must also operate. Baquero et al. [15] introduce delta-CRDTs, compact state-synchronization representations that our Theorem 7 addresses.

The join-semilattice model of CRDTs (Shapiro et al. [1]) provides the algebraic foundation: each CRDT state forms a join-semilattice where the merge function computes the least upper bound. CK-CRDT merge is a *restricted join* — it computes the join within each content-key class independently, then takes the union across classes. This restriction is what enables deduplication: two operations in the same class are joined (one wins), while operations in different classes coexist. The restriction preserves the join-semilattice properties (commutativity, associativity, idempotence) because each class is processed independently. This connection to the algebraic CRDT literature was not explicit in prior work on content-keyed systems.

Recent work strengthens CRDT guarantees and their verification beyond the classical CAI model. Mao et al. [21] observe that eventual consistency is often insufficient for application correctness and introduce *reliable CRDTs*, which layer strongly- or eventually-consistent query guarantees atop CRDT replication; this is orthogonal to and composable with our content-keyed merge, which characterizes *when* content-keying yields convergence. A parallel line of work mechanizes CRDT convergence proofs in proof assistants — Sal [20] provides multi-modal verification of replicated data types in F*/Dafny/Lean, Neem introduces replication-aware specifications, and LeanYjs formalizes Yjs's YATA algorithm in Lean with differential tests against the reference implementation. Our proofs are pen-and-paper; mechanizing the CK-CRDT convergence and no-orphan results (Theorems 1–4) in a proof assistant is natural future work that would place the framework alongside these verified developments.

1.5.2 Content-Addressed Systems IPFS [9] and IPLD use content hashes as identifiers. Git [11] uses SHA-1 hashes of content, tree structure, and parent commits to create an immutable DAG of project history. These systems use content-addressing but are not CRDTs — they have no merge function in the CRDT sense. The Hypercore/Dat protocol [12] uses content-addressed blocks in an append-only log. We classify these as *content-addressed (not CK-CRDT)* — they illustrate the keying pattern but fall outside the CRDT framework.

The closest prior art to CK-CRDTs is Merkle-CRDTs [19], which use Merkle-DAGs — content-addressed directed acyclic graphs — as a transport, persistence, and logical-clock layer for CRDTs, leveraging content addressing to simplify convergent replication over weak messaging layers and very large replica counts. Merkle-CRDTs and CK-CRDTs both combine content-addressing with CRDT merge, but address different problems. Merkle-CRDTs use content hashes to *order, deliver, and persist* operations: content-addressing serves as a clock and transport substrate, while the underlying CRDT (a counter, register, or similar) is unchanged. CK-CRDTs instead use a content-derived key to *partition and deduplicate* operations into equivalence classes, selecting one representative per class; the content key is part of the merge semantics, not merely the transport. Consequently, CK-CRDT merge is a restricted join over content-key classes and — when composed with a downstream CRDT having foreign-key dependencies — requires the no-orphan projection guarantee (Theorem 2) that Merkle-CRDTs do not address, since their setting (opaque immutable blocks) has no entity/edge referential structure. The two approaches are complementary: a CK-CRDT could be transported over a Merkle-DAG layer.

1.5.3 Deduplicating CRDTs Several systems merge concurrent operations by content similarity [2, 3, 7]. Kleppmann [13] addresses Byzantine fault tolerance in CRDTs, showing that content-keying must be verified against adversarial peers — a concern our adversarial test suite (§8.3) directly addresses. Our framework provides convergence conditions that these systems satisfy implicitly, and identifies non-key invariance (K3) as the property they must verify.

1.5.4 Entity Resolution and Record Linkage Fellegi–Sunter [2] and Cohen et al. [3] address record linkage in data integration using probabilistic matching. Christen [16] provides a comprehensive survey of data-matching techniques. LIMES [8] performs large-scale link discovery on the Web of Data. These methods operate in a batch, centralized setting — our CK-CRDT framework extends the keying idea to the distributed, concurrent setting where multiple peers create records independently and must reconcile at merge time without coordination. The connection to record linkage is direct: Fellegi–Sunter's blocking keys are content keys in our terminology, and their matching functions correspond to our κ function. The key difference is that record linkage operates on a fixed database (batch setting), while CK-CRDTs operate on a growing operation log (streaming setting) where new operations arrive concurrently and must be merged without coordination.

1.5.5 Collaborative Editing Yjs [4], Automerge [5], and Loro [6] assign globally unique IDs at creation time, avoiding content-keying at the data-model layer. Google Docs uses server-assigned operation IDs. The substantive reason these systems avoid content-keying is that collaborative text *requires tracking position in a shared sequence* (causal trees, RGA, or list CRDTs) — content-keying would collapse operations at different positions with identical content, breaking the sequence semantics. The dedup-vs-simplicity tradeoff is a secondary effect. At the transport layer, Yjs uses content-hashing for block-level synchronization, but the data model remains ID-at-creation.

---

2. Background and Definitions

2.1 Standard CRDT Model

A CRDT is a data structure that can be replicated across multiple peers, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence requires commutativity, associativity, and idempotence of the merge function (the CAI criteria).

Definition (Convergence). A CRDT converges if for any two peers p_1, p_2 that have received the same set of operations B, applying the merge function M produces identical state: $M_{p_1}(B) = M_{p_2}(B)$.

2.2 Content-Keyed CRDTs: Formal Definition

Let $\mathcal{O}$ denote the operation alphabet (the set of all possible operations) and K the key space. Each operation $o \in \mathcal{O}$ has a set of *content fields* $F_C(o)$ (carrying semantic content — e.g., name, type, description) and *metadata fields* $F_M(o)$ (timestamps, creator IDs, vector clocks). The content-key function κ reads only content fields.

Definition 1 (Content Key). A *content key* is a total function $\kappa : \mathcal{O} \to K$ that depends only on an operation's content fields. The partition $\mathcal{O}/\kappa$ induced by κ defines the equivalence classes under which merge is applied: operations in the same class are merged together; operations in different classes are independent. For each key $k \in K$, $\mathcal{O}_k = \{o \in \mathcal{O} : \kappa(o) = k\}$ is the set of all operations with that key.

Definition 2 (CK-CRDT). A *content-keyed CRDT* is a tuple $(\kappa, \{\rho_k\}, M)$ where: - $\kappa : \mathcal{O} \to K$ is a content-key function. - For each key $k \in K$, $\rho_k : \mathcal{P}(\mathcal{O}_k) \to \mathcal{O}_k$ is a deterministic representative-selection function. For any non-empty finite subset $S \subseteq \mathcal{O}_k$, $\rho_k(S) \in S$ selects one operation as the representative. - $M : \mathrm{Bag}(\mathcal{O}) \to \mathcal{P}(\mathcal{O})$ is the merge function. Given a bag B of operations (a multiset where the same operation may appear from multiple peers), M partitions B into per-key classes $C_k(B) = \{o \in B : \kappa(o) = k\}$, applies ρ_k to each non-empty class, and produces canonical state as the set of representatives:

$$M(B) = \bigcup_{k \in \kappa(B)} \{\rho_k(C_k(B))\}$$

where $\kappa(B) = \{\kappa(o) : o \in B\}$ is the image of the bag under κ.

The state of a CK-CRDT peer is the bag of operations it has received; the merge function M is the canonical projection from bags to state. We assume operations are distinguishable by origin (peer ID + local sequence number), so a bag may contain multiple observations of the same logical operation. Within each class $C_k(B)$, the set of distinct operations is passed to ρ_k.

Definition 3 (Winner Set and Redirect Map). Let B be the operation bag for a CK-CRDT. The *winner set* is $W(B) = \{\rho_k(C_k(B)) : k \in \kappa(B)\}$. An operation $o \in B$ is a *winner* if $o \in W(B)$; otherwise it is a *loser*. The *redirect map* $R : \mathcal{O} \to \mathcal{O}$ maps each loser to its class representative: $R(o) = \rho_{\kappa(o)}(\mathcal{O}_{\kappa(o)} \cap B)$ for losers, and $R(o) = o$ for winners.

Algebraic Structure: Restricted Join-Semilattice. In Shapiro's lattice model [1], a state-based CRDT state forms a bounded join-semilattice $(S, \sqcup)$ with partial order $\leq$. A CK-CRDT state is a *restricted join-semilattice* over equivalence classes $\mathcal{O}/\kappa$: within each class C_k, merge computes the local join $\sqcup_k = \rho_k$; across classes, merge takes the disjoint union $\bigcup_k$. Because each equivalence class is disjoint and ρ_k is commutative, associative, and idempotent (CAI) under argmax selection (Theorem 1), the product structure $(M(B), \bigcup \rho_k)$ forms a valid bounded join-semilattice.

On the choice of ρ_k. The representative-selection function is a *convention*, not a theorem. Any deterministic argmax over a total order on operations yields a valid, convergent CK-CRDT (Theorem 4); the specific choice $\rho_k(S) = \max(S)$ by entity ID in our companion pipeline [14] is a *latest-creation tiebreak* — entity IDs are assigned monotonically at creation time, so the highest ID is the most recently created record. This is the precise meaning of "deduplication by latest-creation reconciliation": it is a selection convention over same-fingerprint duplicates, not a reconciliation of causal age (which is governed by the version-vector partial order in Phase 1).

2.3 Examples

System	Category	Key κ	Representative ρ	Notes
Our pipeline	CK-CRDT	SHA-256(name, type, desc)	max(entity_id)	Canonical example

System	Category	Key κ	Representative ρ	Notes
Deduplicating sync	CK-CRDT	Content hash	First-seen or max-id	
IPFS/IPLD	Content-addressed (not CRDT)	SHA-256(content)	—	Content-addressed storage; no merge function
Syncthing	Content-addressed (not CRDT)	Block hash	—	Block-sync tool; uses file timestamps for conflict resolution
Dat/Hypercore	Content-addressed (not CRDT)	Content hash	—	Append-only log; no merge function
Git (commits)	Content-addressed (not CRDT)	SHA-1(content, tree, parents)	—	Uses 3-way merge, not CRDT. Illustrative only.
Nix (store paths)	Content-addressed (not CRDT)	SHA-256(drv, inputs)	—	Content-addressed package store; merge is not CRDT
Yjs	ID-at-creation	Client-generated clock-based ID	—	Avoids content-keying at data-model layer; uses content-hashing at transport layer
Automerge	ID-at-creation	UUID at creation	—	Same pattern; requires position-tracking
Loro	ID-at-creation	Random ID at creation	—	Same pattern
Google Docs	Centralized	Server-assigned op ID	—	No content-keying needed
Bitcoin (blocks)	Consensus-addressed	SHA-256(block header)	—	PoW consensus, not CRDT merge. Illustrative only.
Ethereum (state)	Consensus-addressed	Keccak-256(state)	—	PoS consensus, not CRDT merge. Illustrative only.

Design insight: The key distinction is *when identity is determined*. In CK-CRDTs, identity is determined by content (at merge time), enabling dedup across independently-created entities. In ID-at-creation systems (Yjs, Automerge, Loro), identity is determined at write time, trading dedup for positional-semantics correctness. Content-addressed storage uses the keying pattern but lacks a CRDT merge function — these are related but not within the framework.

---

3. Theorem 1: Content-Key Monotonicity

Definition 4 (Argmax ρ). A representative-selection function ρ is an *argmax* over a total order $\leq$ on operations if $\rho(S) = \arg\max_{\leq}(S)$ for any non-empty finite S. That is, ρ selects the $\leq$-maximum element of its input set. Any two $\leq$-maximal elements are $\leq$-equivalent, and ρ uses a deterministic tie-breaking rule (e.g., smallest creation timestamp) to select one.

Theorem 1 (Content-Key Monotonicity). Let $(\kappa, \{\rho_k\}, M)$ be a CK-CRDT where each ρ_k is an argmax over a total order $\leq$ on operations (Definition 4). Then:

(a) ρ_k is monotone: for any sets $S \subseteq S' \subseteq \mathcal{O}_k$, $\rho_k(S') \geq \rho_k(S)$.

(b) ρ_k is content-stable: re-import of the canonical representative does not displace it. Formally, $\rho_k(S \cup \{\rho_k(S)\}) = \rho_k(S)$ for any non-empty S.

(c) and (b) are equivalent under the argmax premise.

Proof:

($\Rightarrow$) Suppose ρ_k is monotone. Let $c = \rho_k(S)$. Then $S \cup \{c\} \supseteq S$, so by monotonicity $\rho_k(S \cup \{c\}) \geq \rho_k(S) = c$. But $\rho_k(S \cup \{c\}) \in S \cup \{c\}$, and ρ_k is an argmax over a total order — it selects the unique maximum element (up to $\leq$-equivalence with tie-breaking). The only element of $S \cup \{c\}$ that is $\geq c$ is c itself (since c is already the maximum of S, no element of S exceeds c). So $\rho_k(S \cup \{c\}) = c$. □

($\Leftarrow$) Suppose ρ_k is content-stable: $\rho_k(T \cup \{\rho_k(T)\}) = \rho_k(T)$ for any non-empty T. Let $S \subseteq S' \subseteq \mathcal{O}_k$. We must show $\rho_k(S') \geq \rho_k(S)$. Let $c' = \rho_k(S')$. Since $S \subseteq S'$, we have $\rho_k(S) \in S'$. If $\rho_k(S) \leq c'$, we're done. Suppose for contradiction that $\rho_k(S) > c'$. Then $\rho_k(S) \in S'$ and $\rho_k(S) > c' = \rho_k(S')$. But ρ_k is an argmax over a total order, so $\rho_k(S')$ should be the maximum of S'. Since $\rho_k(S) \in S'$ and $\rho_k(S) > \rho_k(S')$, this is a contradiction. Therefore $\rho_k(S') \geq \rho_k(S)$. Note: this direction follows from the argmax property alone — content-stability is not invoked in the contradiction. The hypothesis is stated for completeness of the equivalence. □

Remark. The equivalence (a) if-and-only-if (b) depends on the argmax property — without it, monotonicity and content-stability are independent. For a non-argmax ρ_k (e.g., random selection), content-stability may hold while monotonicity fails, or vice versa. Theorem 1 applies only to CK-CRDTs with argmax representative-selection.

Corollary 1. In our pipeline, $\rho_k(S) = \max(S)$ (highest entity ID), which is an argmax over the natural total order on IDs. This satisfies monotonicity: $S \subseteq S' \implies \max(S') \geq \max(S)$. This is validated empirically in [14] by `TestMigrationPreservesCanonical`.

---

4. Theorem 2: Layered No-Orphan Invariant

Definition 5 (Foreign-Key Dependency). A downstream CRDT M_{down} has a *foreign-key dependency* on an upstream CK-CRDT M_{CK} if the schema of M_{down}'s operations includes fields whose values are entity IDs produced by M_{CK}.

Notation. Let $R_{\text{id}} : \text{ID} \to \text{ID}$ denote the *canonical redirect function*: for any entity ID e, $R_{\text{id}}(e) = e$ if e is a canonical (winning) entity, and $R_{\text{id}}(e) = e'$ where e' is the canonical entity merged from e otherwise. R_{id} is derived from the redirect map R (Definition 3) by following chains to their fixed point.

Theorem 2 (Layered No-Orphan Invariant — Sufficiency). Let M_{CK} be a CK-CRDT that has been fully merged (the upstream bag B is complete), producing canonical entity IDs via $W(B)$. Let M_{down} be a downstream CRDT with foreign-key dependencies on those IDs. If M_{down} applies R_{id} to all edge endpoints at write time, then every edge endpoint in M_{down}'s output references an entity in $W(B)$ — the no-orphan invariant holds.

Proof:

Assume M_{down} applies R_{id} to all edge endpoints at write time (after the upstream merge is complete). For every edge endpoint e in M_{down}'s output, e has been mapped through R_{id}. If e was a loser ID l, then $R_{\text{id}}(l) = \text{id}(R(o_l))$, which is the ID of a class representative in $W(B)$. If e was already a winner ID, $R_{\text{id}}(e) = e$ and $e \in W(B)$. In both cases, the endpoint references a canonical entity. □

Remark. The converse (necessity) is not unconditional: if no losers exist (each key class has exactly one operation), the invariant holds even without R_{id}. In practice, losers are guaranteed whenever multiple peers independently create operations in the same key-class, which is the primary CK-CRDT use case. Systems requiring strict necessity should verify that at least one key-class has multiple operations.

Corollary 2. In our pipeline, the orphan guard (`crdt_projection.py:494-505`) provides an unconditional version: edges referencing non-canonical entities are dropped, not just redirected. This is strictly stronger than Theorem 2 requires.

Worked example: concurrent edge write during redirect. Consider three peers: Peer A creates entity e_1 (key k_1), Peer B creates entity e_2 (key k_1, same content), and Peer C writes edge $e_3 \to e_2$. At projection time, Phase 2 merges e_1 and e_2 (same fingerprint), selecting $e_1 = \max(e_1, e_2)$ as canonical. The redirect map is $R = \{e_2 \to e_1\}$. Peer C's edge $e_3 \to e_2$ references a loser ID.

- **With durable redirect table** (production path): `resolve_edge_endpoints` queries `kg_entity_redirect` at write time, finds $R(e_2) = e_1$, and writes $e_3 \to e_1$. The edge is rewritten, not dropped.
- **Without durable table** (standalone artifact): the orphan guard drops the edge because $e_2 \notin$ canonical_ids. The edge is lost but no orphan is created.

Both approaches satisfy the no-orphan invariant. The production path is strictly stronger: it preserves the edge by rewriting it, while the standalone artifact preserves the invariant by dropping it. This is the same pattern resolved in Paper 1's production code (§5.4, issue 3).

---

5. Theorem 3: Kernel of CK-Merge

Lemma 1 (Kernel of CK-Merge). The merge function M_{CK} is many-to-one over each equivalence class. The kernel of M_{CK} — the set of operation sets that produce the same canonical output — is exactly the partition into key-classes, provided operations from different key-classes are distinguishable (Assumption D: the merge output carries key tags, or operations are typed by key, so that representatives from different classes can be identified in the output). Under Assumption D, two operation sets O_1, O_2 produce the same $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$ iff for every key-class C_k, the representative $\rho_k(C_k \cap O_1) = \rho_k(C_k \cap O_2)$.

Proof:

($\Rightarrow$) If $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$, then the output sets are equal: $\{\rho_k(C_k \cap O_1) : k \in \kappa(O_1)\} = \{\rho_k(C_k \cap O_2) : k \in \kappa(O_2)\}$. For any key $k \in \kappa(O_1) \cap \kappa(O_2)$, the representative $\rho_k(C_k \cap O_1)$ appears in both output sets, and since ρ_k is deterministic and $\rho_k(C_k \cap O_1) \in C_k$, the only element of the output set in class C_k is $\rho_k(C_k \cap O_1)$. Therefore $\rho_k(C_k \cap O_1) = \rho_k(C_k \cap O_2)$. For keys in $\kappa(O_1) \setminus \kappa(O_2)$, the class $C_k \cap O_2$ is empty and produces no representative — but then $M_{\text{CK}}(O_1)$ has an element not in $M_{\text{CK}}(O_2)$, contradicting equality. So $\kappa(O_1) = \kappa(O_2)$ and all representatives match.

($\Leftarrow$) If for every key k, the representative is the same, then $M_{\text{CK}}(O_1)$ and $M_{\text{CK}}(O_2)$ produce identical representative sets by definition of M_{CK}. $\square$

Information-loss corollary. The information discarded by CK-CRDT merge is exactly the within-class loser set $O \setminus W(O)$, where $W(O) = \{\rho_k(C_k) : k \in \kappa(O)\}$. This is not recoverable from the canonical state: for any class C, any subset $C' \subseteq C$ may be designated losers and the merge output is invariant.

Lemma 2 (Information-loss lower bound). Any CK-CRDT merge $(\kappa, \{\rho_k\}, M)$ satisfying (K1)–(K3) must discard at least $|O| - |\kappa(O)|$ operations. This bound is tight: the CK-CRDT merge achieves exactly this loss.

Proof. We show two things: (1) the loss is at least $|O| - |\kappa(O)|$, and (2) CK-CRDT merge achieves exactly this.

(1) **Lower bound.** The canonical state Σ contains at most one representative per key class. Since there are $|\kappa(O)|$ distinct keys, $|\Sigma| \leq |\kappa(O)|$. Therefore the number of discarded operations is at least $|O| - |\kappa(O)|$.

(2) **Tightness.** CK-CRDT merge produces exactly $|\kappa(O)|$ representatives (one per non-empty class), discarding exactly $|O| - |\kappa(O)|$ operations. No K1-K3-compliant merge can do better: by Definition 2, ρ_k maps each key class to a single element (the representative). This is a structural requirement of the CK-CRDT definition — the merge function selects exactly one representative per class by construction. K1 and K2 ensure the partition into classes is deterministic and content-local, but the “one representative per class” constraint comes from the definition of ρ_k, not from the key properties. Therefore any K1-K3 merge discards at least $|O| - |\kappa(O)|$ operations, and the CK-CRDT merge achieves exactly this. $\square$

Corollary 3. The information-loss bound $|O| - |\kappa(O)|$ is independent of the specific merge function ρ_k. Any two CK-CRDT merges satisfying (K1)–(K3) with the same key function κ discard the same number of operations, though they may discard different operations (different winners).

---

6. Theorem 4: Content Key Properties

We define three properties of the content key κ. Each operation o has a set of *content fields* $F_C(o)$ (e.g., name, type, description) and *metadata fields* $F_M(o)$ (timestamps, creator IDs). The content-key function κ reads only content fields. *Key-relevant fields* are the subset of F_C that κ depends on; *non-key fields* are the rest of $F_C \cup F_M$.

(K1) Determinism: κ is a deterministic function of the operation: $\kappa : \mathcal{O} \to K$. The same operation always produces the same key on every peer. Formally: for any operation o and any two peers p_1, p_2 that have received o, $\kappa_{p_1}(o) = \kappa_{p_2}(o)$.

(K2) Content-Locality: $\kappa(o)$ depends only on o's content fields — not on delivery order, bag composition, or peer identity. Formally: for any operation o and any two bags B_1, B_2 both containing o, $\kappa_{B_1}(o) = \kappa_{B_2}(o)$.

(K3) Non-Key Invariance: If operation o' extends o by updating at least one non-key field and does not update any key-relevant field, then $\kappa(o') = \kappa(o)$. This ensures that a metadata update (which changes non-key fields under LWW) does not change the key.

Theorem 4 (Content Key Properties — Necessity and Sufficiency). If κ satisfies (K1)–(K3) and each ρ_k is an argmax over a total order (Definition 4), then M converges: all peers with the same operation bag B produce the same canonical state.

Moreover, each of (K1)–(K3) is necessary for this guarantee: violating (K1) or (K2) permits divergence (different peers produce different outputs), and violating (K3) permits correctness degradation (convergence holds but entity deduplication fails). Thus (K1)–(K3) are jointly necessary and sufficient for convergent deduplication under argmax selection.

Proof (sufficiency):

(K1) ensures all peers compute the same key for each operation, hence the same partition $\kappa(B)$ for any bag B. (K2) ensures $\kappa(o)$ is the same regardless of which other operations have been delivered — the partition is invariant under delivery order. (K3) ensures that a metadata update does not change the key.

Given a stable partition, convergence follows from two facts:

(1) Define a *binary merge* m_k on each key class k as $m_k(o_1, o_2) = \rho_k(\{o_1, o_2\})$. Since ρ_k is an argmax over a total order:

- *Commutativity:* $m_k(o_1, o_2) = \rho_k(\{o_1, o_2\}) = \rho_k(\{o_2, o_1\}) = m_k(o_2, o_1)$.
- *Associativity:* $m_k(m_k(o_1, o_2), o_3) = \rho_k(\{\rho_k(\{o_1, o_2\}), o_3\}) = \rho_k(\{o_1, o_2, o_3\}) = m_k(o_1, m_k(o_2, o_3))$ because the argmax of a set depends only on the set, not on the order of pairwise reduction.
- *Idempotence:* $m_k(o, o) = \rho_k(\{o\}) = o$, and by Theorem 1(b), $m_k(\rho_k(S), \rho_k(S)) = \rho_k(S)$.

Extending m_k to finite sets by iterated application: $m_k(\{o_1, \dots, o_n\}) = \rho_k(\{o_1, \dots, o_n\})$, since the argmax of a set is well-defined independent of reduction order.

(2) Since κ provides a stable partition (by K1–K3), the bag B decomposes into independent per-key classes $C_k(B)$. The merge function $M(B) = \bigcup_k \{\rho_k(C_k(B))\} = \bigcup_k \{\text{iterated-}m_k(C_k(B))\}$ is a disjoint union of independent per-class merges, each of which satisfies CAI. The union of independent CAI merges is itself CAI: commutativity and associativity hold because classes are disjoint and processed independently; idempotence holds because each class is idempotent.

Therefore all peers with the same bag B produce the same $M(B)$. □

Proof (necessity). Each of (K1)–(K3) is necessary for the convergent-deduplication guarantee, in the precise sense that violating it (while satisfying the other two) admits an operation bag for which the guarantee fails. (Violating a property need not break convergence for *every* bag, but it means the universal guarantee no longer holds — there exists a bag for which it fails.) We establish necessity by counterexample for each property.

Violating (K1) can break convergence. If κ is non-deterministic, the same operation o may receive different keys on different peers. Construct a bag $B = \{o, o'\}$ where on peer A, $\kappa_A(o) = \kappa_A(o') = k_1$ (same class), while on peer B, $\kappa_B(o) = k_1$, $\kappa_B(o') = k_2$ (different classes). Peer A computes $M_A(B) = \{\rho_{k_1}(\{o, o'\})\}$ (a single representative). Peer B computes $M_B(B) = \{\rho_{k_1}(\{o\}), \rho_{k_2}(\{o'\})\} = \{o, o'\}$ (two representatives). If $\rho_{k_1}(\{o, o'\})$ selects a single element (which it must — ρ_{k_1} returns one operation), then $|M_A(B)| = 1 \neq 2 = |M_B(B)|$, violating convergence. □

Violating (K2) can break convergence. Let $\kappa(o)$ depend on the bag B: $\kappa_B(o) = k_a$ if $|B| = 1$ and $\kappa_B(o) = k_b$ if $|B| > 1$. This violates content-locality (K2 requires κ to be a function of the operation alone). Consider two peers that both end up with bag $B = \{o_1, o_2\}$ via different delivery orders. Peer A receives o_1 first: when alone, $\kappa(o_1) = k_a$; after o_2 arrives, $\kappa(o_1)$ becomes k_b. Peer B receives o_2 first: symmetric reasoning. The final $\kappa(o_1)$ values differ across peers for the same final bag, producing different partitions and different $M(B)$. □

Violating (K3) causes correctness degradation. Let o have key-relevant fields ($name$ = "alice", $type$ = "person") and non-key field $description$ = "". Let o' extend o with $description$ = "lawyer". Define $\kappa(o) = k_1$ but $\kappa(o') = k_2$ (the key derivation reads description, violating K3). Both peers have bag $B = \{o, o'\}$. $M(B) = \{\rho_{k_1}(\{o\}), \rho_{k_2}(\{o'\})\} = \{o, o'\}$ — two separate entities despite o and o' representing the same real-world entity with different descriptions. This is a *semantic duplicate*: the same entity appears twice in the output. While convergence (same-bag-same-output) holds, the CK-CRDT has failed its primary purpose — entity deduplication. □

Empirical verification. All three necessity counterexamples are verified by `test_adversarial.py::TestNecess` (4 tests): `test_k1_necessity_divergence` confirms $|M_A(B)| = 1 \neq 2 = |M_B(B)|$; `test_k2_necessity_divergen` confirms different delivery orders produce different representative sets; `test_k3_necessity_semantic_duplicate` confirms 2 entities instead of 1 when K3 is violated. The sufficiency direction is verified by `test_k1_k3_sufficiency_convergence`: all 24 permutations of 4 operations produce identical output when K1–K3 hold.

Boundary of the content-only requirement. The content-key requirement (K2: content-locality) says κ must be a function of the operation's content fields alone. This is sufficient for CK-CRDTs but not necessary for all CRDTs. Consider a G-Counter CRDT: each increment operation carries a peer ID and a clock value, and the merge function must read these external references (peer IDs, clocks) to compute the component-wise maximum. The G-Counter satisfies CAI but violates K2 — its merge outcome depends on metadata fields (peer IDs) that are not inherent content. The CK-CRDT framework does not apply to such CRDTs; it characterizes the specific subclass where content is the sole basis for partitioning and representative selection.

---

7. Classification of Real Systems

We classify existing systems into CK-CRDT instances, ID-at-creation systems, and non-CRDT content-addressed systems.

7.1 Classification Criteria

1. **Identity mechanism:** Content-keyed (CK), ID-at-creation (ID), or Consensus-addressed (CS).
2. **Merge function:** CRDT merge (partition + select / LWW), state union, or manual resolution.
3. **No-orphan invariant:** Native to merge or application-managed.

7.2 Classification Table

System	Category	Key κ	(K1)	(K2)	(K3)	Notes
Our pipeline	CK-CRDT	SHA-256(name, type, desc)	Y	Y	Y	Canonical example; max-ID representative
Deduplicating sync	CK-CRDT	Content hash	Y	Y	Y	First-seen or max-id selection
IPFS/IPLD	Content-addressed	SHA-256(content)	Y	Y	Y*	*K3 vacuous (content immutable); no CRDT merge. Not a CK-CRDT.
Git (commits)	Content-addressed	SHA-1(content, tree, parents)	Y	Y	Y*	*K3 vacuous (commits immutable); uses 3-way merge. Not a CK-CRDT.
Syncthing	Content-addressed	Block hash	Y	Y	Y*	Block-sync tool; conflict resolution via file timestamps
Dat/Hypercore	Content-addressed	Content hash	Y	Y	Y*	Append-only log; no merge function
Nix (store paths)	Content-addressed	SHA-256(drv, inputs)	Y	Y	Y*	Content-addressed package store

System	Category	Key κ	(K1)	(K2)	(K3)	Notes
Bitcoin (blocks)	Consensus-addressed	SHA-256(block header)	Y	Y	Y*	PoW consensus, not CRDT. Illustrative only.
Ethereum (state)	Consensus-addressed	Keccak-256(state)	Y	Y	Y*	PoS consensus, not CRDT. Illustrative only.
Yjs	ID-at-creation	Client-generated clock-based ID	—	—	—	Position-tracking requires ID-at-creation. Uses content-hashing at transport layer.
Automerge	ID-at-creation	UUID at creation	—	—	—	Same pattern; sequence CRDT
Loro	ID-at-creation	Random ID at creation	—	—	—	Delta-CRDT with ID-at-creation
Google Docs	Centralized	Server-assigned op ID	—	—	—	No content-keying needed
VS Code Live Share	Centralized	Session-scoped IDs	—	—	—	Session-scoped; duplicates acceptable

Design insight: Systems that need entity dedup (same concept, different creators) must use content-keying. Collaborative editors *cannot* use content-keying for position-dependent operations because position is part of identity — content-keying would incorrectly collapse operations at different positions. The dedup-vs-simplicity tradeoff arises from this structural constraint, not a design choice.

Third-party instantiation: Docker/OCI image layer deduplication. Docker's storage driver deduplicates image layers across images using content-addressed hashing (SHA-256 of the layer tarball). When two images share a layer — same filesystem contents, different build contexts — only one copy is stored on disk. This is a CK-CRDT instance: the content key κ is the SHA-256 hash of the layer content; (K1) holds because identical layer contents produce identical hashes;

(K2) holds because the hash depends only on the layer's filesystem content, not on which image references it or when it was built; (K3) is vacuous because layers are immutable once created. The "merge" selects one representative per key class (the stored layer) and discards duplicates — exactly the CK-CRDT pattern. Docker did not design this as a CK-CRDT; the framework classifies it post-hoc. This is the intended use of the formalism: not to prescribe design, but to illuminate structure that already exists in systems built independently. Note that Docker's layer eviction (reference-count GC when images are removed) is a non-tombstone membership model, unlike the 2P-Set semantics in entity dedup. The framework is agnostic to membership structure: K1–K3 constrain the content key, not how members are added or removed from a key class.

What Docker doesn't do that CK-CRDTs do. Docker's dedup is passive: identical layers are stored once, but there is no merge operation, no redirect map, and no convergence guarantee across independent registries. If two Docker Hub mirrors independently build the same layer, they store one copy each — there is no mechanism to reconcile them. A CK-CRDT formulation would add: (1) a redirect map that records which layer IDs are equivalent, enabling cross-registry dedup after sync; (2) a convergence guarantee: two mirrors that start with the same layers and receive the same build operations will converge to the same canonical state; (3) an orphan invariant: no image manifest references a layer that doesn't exist in the canonical store. These are exactly the properties our knowledge-graph pipeline provides (§5 of [14]). Docker doesn't need them because it operates in a single-registry model; multi-registry sync (Docker Hub <-> ECR <-> GCR) would.

Nix as a borderline case. Nix content-addressed store paths are computed from derivation inputs; two identical builds produce the same path. This is the pure keying pattern, and Nix *does* have a merge-like operation (channel merging, template instantiation). It illustrates the framework's boundaries: the keying pattern holds, but whether the merge is a CK-CRDT depends on the specific merge implementation.

System	Class	Key Function κ	Representative Selection ρ	Merge Type	No-Orphan Support
Our KG Pipeline	CK-CRDT	SHA-256(name, type, desc)	max(entity_id)	2P-Set + LWW	Yes (Phase 3 rewrite)
IPFS	Content-Addressed	Multihash (SHA-256, etc.)	N/A (immutable)	Union	N/A
Git	Content-Addressed	SHA-1 / SHA-256	N/A (immutable commits)	3-way merge	Manual
Syncthing	CK-CRDT	File path + block hashes	Latest modification time	LWW	N/A
Yjs	ID-at-creation	Client ID + clock	N/A (positional)	Y-Array / Y-Map	No
Automerge	ID-at-creation	Actor ID + counter	N/A (positional)	B-tree / Op-log	No
Loro	ID-at-creation	Peer ID + counter	N/A (positional)	Movable Tree	No

8. Discussion

8.1 When to Use Content-Keying

Content-keying is necessary when: - Multiple peers can create semantically identical entities independently. - The system must collapse duplicates at merge time. - Entity identity is derived from content, not assigned at creation.

Content-keying is optional when: - Entities are assigned globally unique IDs at creation (Yjs, Automerge). - Duplicates are acceptable (collaborative whiteboards, append-only logs). - A higher-level reconciliation step handles dedup (data integration pipelines).

8.2 Connection to Prior Work

This paper generalizes the three-phase knowledge-graph projection pipeline described in Sadhu [14]. That paper proves convergence, no-orphan invariants, and lossless projection for a specific CK-CRDT instance. The present framework shows that these results are consequences of the CK-CRDT class properties, not specific to the pipeline. Theorem 1 generalizes the Selection Convention of [14] (latest-creation winner via max(id); [14] had originally labelled this "Theorem 2 (Canonical-Id Monotonicity)" but it is a convention, not a theorem); Theorem 2 generalizes Corollary 1 of [14] (unconditional no-orphan); Theorem 3 generalizes the lossless-projection / kernel result of [14] (Main Result 3, stated there as Lemmas 1–2); Theorem 4 generalizes Theorem 3 of [14] (Content Key Properties — Necessity and Sufficiency), lifting the pipeline-specific convergence conditions to the framework level.

Connection to the vv_sum corrigendum. Sadhu [14] corrected vv_sum → vv_dominates for edge merge. The underlying issue was that vv_sum conflated concurrent vectors, violating the monotonicity properties of the ordering: a causal update (bumping one peer's clock) could change the sum in a way that reversed the ordering. vv_dominates respects the causal partial order — a causal update can only strengthen dominance, never reverse it — so the ordering is monotonic under causal updates. In CK-CRDT terms, vv_sum violated K2 (content-locality) because the merge outcome depended on the bag's vector-clock composition, not on operation content alone.

8.3 Limitations

- **Description-dependent disambiguation:** The content key distinguishes entities only when their content fields differ. Two entities with identical content fields merge even if they represent different concepts — the key does exactly what it is defined to do, but content representations may lack distinguishing fields.
- **Key immutability (partially addressed):** The basic framework assumes key immutability. Theorem 6 extends this to adaptive keys under an acyclic migration graph.
- **Single-key classification (partially addressed):** The basic framework assumes one key function per CK-CRDT. Theorem 4 extends this to composite keys.
- **Deletion and tombstones:** The framework has no explicit model for deletion. Standard CRDTs handle deletion via tombstones; a CK-CRDT must define whether a tombstone carries the same content key as the original entity. If a tombstone has the same key, re-creation of the same entity after deletion could produce conflicting merge results. We assume deletions

are handled outside the CK-CRDT layer (e.g., via observed-remove semantics) — this is an area for future work.

- **Cross-key causal dependencies:** The framework assumes operations in different key-classes are independent. In knowledge graphs, entity creation (class A) and edge referencing (class B) have causal dependencies. Theorem 2 addresses one aspect of this (foreign-key redirect), but a full treatment of cross-class causality is open.
- **Total-order construction:** Theorem 1 and Theorem 4 require each ρ_k to be an argmax over a total order. Constructing a total order that all peers agree on is a separate distributed-systems problem. The framework is agnostic to the specific clock construction — any total order that peers agree on satisfies the requirement. In practice, hybrid logical clocks [18] provide a total order that respects causality and is robust to clock skew; Lamport timestamps [17] provide a simpler alternative when causal ordering is sufficient. The choice of clock is an engineering decision outside the scope of this framework.
- **Partial replication:** The convergence proof (Theorem 4) assumes all peers have the same bag B. In practice, peers reconcile with different subsets under eventual consistency. The framework assumes that all peers eventually converge to the same bag — if the synchronization protocol ensures complete delivery of all operations (or anti-entropy reconciliation), strong eventual consistency follows. Because the merge is set-deterministic, commutative, and idempotent rather than associative across arbitrary partial-bag subsets, strong eventual consistency relies on complete-delivery transport guarantees rather than intermediate partial-summary merges. Analysis under partial-replication models (e.g., gossip-based sync) is future work at the *proof* level; the reference implementation *demonstrates* reconciliation dynamics empirically via `test_partial_replication_convergence` (Appendix B.3): peers seeded with distinct op subsets project on their partial bags, then receive missing operations and re-project, reaching the identical canonical state as a peer holding the full bag throughout.
- **Scalability:** CK-CRDT merge is $O(N)$ for partitioning N operations by key plus $O(M)$ for representative selection across M distinct keys. When $M \approx N$ (no dedup), the framework provides no benefit over standard set-based CRDTs. The approach is most beneficial when $M \ll N$ (high dedup ratio). Empirical measurements on the reference implementation (`ck_crdt.py`, pure in-memory merge + dedup, no SQLite I/O) at K=1000 (realistic workload) show throughput of 287K ops/s at 100K, 235K at 1M, and 151K at 10M — a 1.9x degradation attributable to Python runtime overhead, not algorithmic. Paper 1's implementation (`crdt_projection.py`) shows comparable throughput (271K→138K at K=1000), confirming $O(N)$ scaling in both cases. At 575 bytes/op, 10M ops requires ~5.8 GB memory.
- **Unicode canonicalization.** The reference implementation applies Unicode NFKC normalization (handles NBSP, ligatures, compatibility equivalents), strips format characters (ZWSP, RTL marks, BOM), and collapses whitespace. Smart quotes (`\u201C`/`\u201D`) are preserved as meaningful punctuation. Agents using different Unicode normalizations may still produce different fingerprints if their canonicalizers disagree on edge cases. Verified empirically via adversarial tests (`test_adversarial.py::test_nfkc_normalization_covers_unicode_whitespace`).
- **Adversarial robustness (verified).** The framework was tested against 36 adversarial scenarios across 11 categories including timestamp skew, Byzantine version vectors, 10K operations with colliding fingerprints, K1-necessity counterexamples, adaptive key migration, and — new in this revision — partial-replication convergence (Appendix B.3). The CK-CRDT merge degrades gracefully under adversarial input — a 10,000-operation single-fingerprint group merges in <0.1s without crash or data corruption. The K1-necessity counterexample (two peers using different normalizations producing different fingerprints) confirms that K1 is both necessary and sufficient for convergence. See `test_adversarial.py` for the full suite.

8.4 Where the Framework Breaks

The CK-CRDT framework has three structural failure modes. **First, content-key collisions across unrelated entities.** If two genuinely distinct entities happen to share identical `(name, type, description)` fields — "Java" (the programming language) and "Java" (the island) — the framework merges them. This is not a bug; it is a fundamental limitation of content-keying. The framework can detect the collision (different `entity_ids`, same fingerprint) but cannot resolve it without external disambiguation. Systems that need to distinguish such homonyms must extend the content key with additional fields (e.g., source document, mention context) — moving toward Theorem 5's composite keys. **Second, causal dependencies across key classes.** The framework assumes operations in different key-classes are independent. In practice, entity creation (class A) and edge referencing (class B) have causal dependencies: an edge cannot exist without its endpoints. Theorem 2 addresses this for the specific case of foreign-key redirects, but a general treatment of cross-class causality is open. A system that violates this assumption (e.g., processing edges before entities are projected) may produce transient orphans that the framework cannot prevent. **Third, adaptive key cycles.** Theorem 7 proves that adaptive keys converge only when the migration graph is acyclic. If a key migration function creates a cycle (entity A's key migrates to B, B's key migrates back to A), convergence is not guaranteed. In practice, key migrations are rare andmonitored; cycles indicate a bug in the migration logic, not a framework limitation. But the framework provides no automatic detection or prevention.

8.5 Open Problems

1. **CK-CRDTs with partial-order ρ.** Theorem 1 requires a total order for the argmax. Can content-stability or monotonicity be defined and proven for ρ operating over partial orders (e.g., vector clocks)?
2. **Adversarial key collisions.** A malicious peer could craft operations to deliberately collide with or avoid existing keys. What security properties must κ satisfy beyond determinism and locality? ### 8.6 Practitioner Guidelines

- **Use CK-CRDT when:** Concurrent independent creation of semantically identical entities is expected across peers, and automatic deduplication is required without central coordination.
- **Use ID-at-creation when:** Sequence or tree position is part of operation identity (rich text, collaborative editing), or when identity cannot be derived deterministically from content fields.

8.7 Summary of Extensions

We address the four questions from §1.2. Theorems 5 and 6 follow directly from Theorem 4. Theorems 7 and 8 are sufficient conditions under their stated assumptions.

9. Extensions

9.1 Theorem 5: Multi-key CK-CRDTs

Theorem 5 (Multi-key CK-CRDTs). Let $\kappa' = (\kappa_1, \kappa_2)$ be a composite content key where $\kappa_1 : \mathcal{O} \to K_1$ and $\kappa_2 : \mathcal{O} \to K_2$ are component keys. If each κ_i satisfies (K1)–(K3) individually, then κ' satisfies (K1)–(K3) and the CK-CRDT (κ', ρ) converges.

Proof:

Let $\kappa'(o) = (\kappa_1(o), \kappa_2(o))$ where $\kappa_i : \mathcal{O} \to K_i$ are component keys.

(K1) for κ': Since $\kappa'(o) = (\kappa_1(o), \kappa_2(o))$ is a function of o alone (each κ_i is deterministic by K1), κ' is deterministic by construction. The same operation always produces the same composite key on every peer.

(K2) for κ': Since each $\kappa_i(o)$ depends only on o's content fields (by (K2) for each component), $\kappa'(o)$ also depends only on o's content fields. Therefore κ' is invariant under delivery order.

(K3) for κ': If o' extends o by updating only non-key fields (under κ''s key-relevant field set, which is the union of κ_1's and κ_2's key-relevant fields), then $\kappa_i(o') = \kappa_i(o)$ for each i (by (K3) for each component), so $\kappa'(o') = \kappa'(o)$. □

Corollary 4. Our pipeline's fingerprint key $\kappa(o) = \text{SHA-256}(\text{name}, \text{type}, \text{description})$ is a composite key with three components. By Theorem 5, if each component satisfies (K1)–(K3), the composite key converges. Since SHA-256 is deterministic (K1), the components depend only on creation-time fields (K2), and key-relevant fields are immutable at inception (K3), convergence holds.

9.2 Theorem 6: Deterministic Approximate Keys

Theorem 6 (Deterministic approximate keys). Let $\kappa : \mathcal{O} \to K$ be an approximate content key (e.g., based on Levenshtein distance or Jaccard similarity). If κ is deterministic — same inputs produce the same key — then the CK-CRDT (κ, ρ) converges. If κ is non-deterministic (same inputs produce different keys on different peers), convergence fails by (K1) violation.

Proof: If κ is deterministic, (K1) holds by definition. For (K2): the similarity metric operates on the operation's content fields as its sole input — peer-local reference sets are not used in κ computation. For (K3): a metadata update that does not change content fields leaves the similarity between any two operations unchanged. Convergence follows from Theorem 4. If κ is non-deterministic, (K1) is violated, and convergence may fail (per the K1 violation construction in §6). □

Corollary 5. Fuzzy record linkage systems (e.g., those using Levenshtein distance with a threshold) satisfy the CK-CRDT convergence conditions iff the similarity computation is deterministic and peer-independent. Most standard implementations are deterministic (same strings → same distance), so convergence holds. Systems using randomized algorithms or peer-local reference sets violate (K1) or (K2) and may diverge.

9.3 Theorem 7: Adaptive Keys

Definition 6 (Key Migration Graph). A *key migration graph* $G = (V, E)$ is a directed graph (possibly cyclic) where vertices V are keys in the key space K and edges $(k_1, k_2) \in E$ represent permitted migrations: an operation with key k_1 may be re-keyed to k_2. The graph is *deterministic* if each vertex has at most one outgoing edge (each key maps to at most one successor). Migration triggers on every merge application.

Theorem 7 (Adaptive Keys — Sufficiency). Let $(\kappa, \{\rho_k\}, M)$ be a CK-CRDT whose key function κ evolves according to a deterministic key migration graph G. If G is acyclic, then the CK-CRDT converges. If G contains a cycle, convergence may break — different peers may compute different final keys for the same operation, violating (K1).

Proof:

(⇒) Suppose G is acyclic. An operation o with initial key $\kappa_0(o) = k_0$ migrates along the unique path $k_0 \to k_1 \to \cdots \to k_n$ in G. Since G is acyclic and deterministic (at most one outgoing edge per vertex), the path is finite, has length at most $|V| - 1$, and terminates at a unique sink vertex k_n (no outgoing edges). The final key $\kappa_n(o) = k_n$ is well-defined and independent of migration order — it is the unique sink reachable from k_0 in G. Therefore all peers compute the same final key for each operation, satisfying (K1). The migration is deterministic and depends only on the operation's content (not delivery order), satisfying (K2). The migration terminates at a sink after at most $|V| - 1$ steps, so no further updates change the key, satisfying (K3). Convergence follows from Theorem 4.

(⇐) Suppose G contains a cycle $k_0 \to k_1 \to \cdots \to k_0$. An operation o with initial key k_0 would migrate on every merge application, cycling indefinitely and never reaching a stable key. Two peers that have applied merge different numbers of times to the same initial operation (due to different delivery histories) will have different κ values for that operation, violating (K1). Note: if the migration function is bounded (e.g., time-to-live counters or history-dependent stopping conditions), cycles may converge — the framework's acyclicity condition applies to unbounded, deterministic migration. □

Corollary 6. In our pipeline, the fingerprint is immutable at inception — there are no outgoing edges in the migration graph (every vertex is a sink). This is the trivially acyclic case. A system that allows fingerprint re-computation (e.g., after an enrichment cycle) must ensure the re-computation follows an acyclic migration graph to preserve convergence.

9.4 Theorem 8: Delta-CRDT Composition

Theorem 8 (Delta-CRDT Composition — Sufficiency). Let $(\kappa, \{\rho_k\}, M)$ be a CK-CRDT and let $\delta : S \to \Delta$ be a delta-computation function that computes a compact representation of the state transition, where S is the set of canonical states (subsets of $\mathcal{O}$). We say δ is *stratified* if it depends only on $M(B)$ (the merge output), not on B directly. If δ is stratified, then the composition $\delta \circ M$ preserves convergence.

Proof:

If δ depends only on $M(B)$, then for any two bags B_1, B_2 with $M(B_1) = M(B_2)$, we have $\delta(M(B_1)) = \delta(M(B_2))$. Since M converges (by Theorem 4, assuming κ satisfies (K1)–(K3)), the composition $\delta \circ M$ also converges: all peers with the same bag produce the same delta.

---

10. Conclusion

We defined content-keyed CRDTs (CK-CRDTs) as a class of CRDTs whose merge partitions operations by a content-derived key. We proved four main results:

1. **Content-Key Monotonicity** (Theorem 1): for argmax representative-selection, monotonicity and content-stability are equivalent — the structural foundation for CK-CRDT merge.
2. **Layered No-Orphan Invariant** (Theorem 2): canonicalization at write time ensures no-orphan guarantees under downstream CRDTs with foreign-key dependencies.
3. **Information-Loss Characterization** (Lemmas 1–2): the information discarded is exactly the within-class loser set, and this is a tight lower bound — no K1-K3-compliant merge can discard fewer operations.

4. **Content Key Properties** (Theorem 4): determinism, content-locality, and non-key invariance are necessary and sufficient for convergent deduplication under argmax selection — (K1) and (K2) are necessary for convergence (violating either permits divergence) and (K3) is necessary for correct deduplication (violating it permits semantic duplicates), and together they are sufficient.

We additionally established (§9) that composite keys inherit convergence (Theorem 5), approximate keys converge when deterministic (Theorem 6), adaptive keys converge under acyclic migration (Theorem 7), and CK-CRDTs compose with delta-CRDTs under stratified computation (Theorem 8). The framework classifies Docker, IPFS, Git, Yjs, Automerge, and Loro, explaining why ID-at-creation systems avoid content-keying (position-tracking requires ID-at-creation) and when content-keying is necessary (multiple peers creating semantically identical entities independently).

The framework classifies content-addressed systems, version control, deduplicating sync, collaborative editors, and our knowledge-graph pipeline. It explains why ID-at-creation systems (Yjs, Automerge) avoid content-keying (position-tracking requires ID-at-creation; the dedup capability would break sequence semantics) and when content-keying is necessary (when multiple peers create semantically identical entities independently).

Appendix A: Worked Comparison with ID-at-Creation Systems

We compare CK-CRDT merge with Yjs and Automerge on a concrete problem: two agents independently create the same entity and add edges to it.

Setup. Agent A creates entity "alice" (ID=42) with type "person" and adds edge (42→15, "knows"). Agent B independently creates "alice" (ID=99) with type "person" and adds edge (99→30, "works_with"). The two agents then sync.

A.1 CK-CRDT Merge (Our Framework)

Phase 1 (Entity merge): Both ops are `add` operations for entities with content `(name=alice, type=person)`. The content key κ computes the same fingerprint for both. Merge selects the LWW winner by version vector: neither dominates (disjoint peers), so timestamp breaks the tie. Agent B's op (t=200) wins. Result: entity 99 survives, entity 42 is a loser.

Phase 2 (Dedup + redirect): Both entities share fingerprint → redirect map `{42: 99}`.

Phase 3 (Edge redirect): Edge (42→15) is rewritten to (99→15). Edge (99→30) passes through.

Final state: 2 canonical entities (99/alice, 15/bob), 2 edges ((99→15), (99→30)). No orphans.

A.2 Yjs Merge

Yjs assigns a client-generated clock-based ID at creation. Agent A creates `Y.Map()` with ID `clientA:1`. Agent B creates `Y.Map()` with ID `clientB:1`. These are distinct Yjs objects — Yjs has no mechanism to detect they represent the same entity.

Final state: 3 objects: `clientA:1` (alice), `clientB:1` (alice), and whatever edges exist. No dedup occurs. The application layer must implement entity resolution separately.

What Yjs does well: Position-preserving text editing, real-time collaboration, offline support. Yjs is optimized for the case where entities are unique by construction (each character position has a unique ID). Content-keying would break sequence semantics.

A.3 Automerge Merge

Automerge uses UUIDs assigned at creation. Agent A creates `{_id: "uuid-a", name: "alice", type: "person"}`. Agent B creates `{_id: "uuid-b", name: "alice", type: "person"}`. Automerge merges by per-field LWW using actor IDs.

Final state: Automerge sees two distinct objects (different `_ids`). It does not deduplicate. The document contains both `uuid-a` and `uuid-b` with identical content. Automerge's merge is correct (no data loss) but does not collapse duplicates.

What Automerge does well: JSON document editing, field-level LWW, portable snapshot format. Like Yjs, it assumes entities are unique by construction.

A.4 Summary

Property	CK-CRDT	Yjs	Automerge
Entity dedup	Yes (content key)	No (ID-at-creation)	No (ID-at-creation)
Redirect map	Yes	No	No
No-orphan invariant	Yes (Theorem 2)	No (application must ensure)	No (application must ensure)
Position tracking	No (not designed for it)	Yes (optimized for it)	Yes (supported)
Offline sync	Yes (CRDT)	Yes (CRDT)	Yes (CRDT)
Convergence guarantee	Yes (Theorems 1, 4)	Yes (CRDT)	Yes (CRDT)

The tradeoff is structural, not qualitative. CK-CRDTs and ID-at-creation systems solve different problems. CK-CRDTs are necessary when multiple peers create semantically identical entities independently and the system must collapse duplicates at merge time. ID-at-creation is necessary when entities are unique by construction and position-tracking requires stable identities. The framework (§8.1) identifies exactly when each approach is appropriate.

Quantitative comparison. Paper 1 [14] §7.1 reports a head-to-head benchmark: on 5,000 concurrent entity ops with 50 distinct entities, the naive merge (ID-at-creation semantics) produces 4,950 duplicates and 460 orphan edges; the CK-CRDT pipeline deduplicates to 50 canonical entities with zero orphans at 38% overhead — dominated by Phase 1 entity merge (94% of runtime), not by content-keyed dedup.

Appendix B: Cross-Paper Instantiation Map

This appendix shows how Paper 1's three-phase pipeline [14] is a concrete instantiation of Paper 2's abstract CK-CRDT framework. Every claim in Paper 1 maps to a specific theorem or property in Paper 2. This is the "third adopter" test: the framework predicts behavior of a system built independently.

B.1 Entity-Level Mapping

Paper 1 (Pipeline)	Paper 2 (Framework)	Mapping
Content key: `SHA-256(name, type, desc)`	$\kappa : \mathcal{O} \to K$	$\kappa(o) =$ fingerprint$(o.\text{name}, o.\text{type}, o.\text{desc})$
`compute_fingerprint()`	K1 (determinism)	Same inputs $\to$ same hash; verified by `test_k1_collision_resistance`
Fingerprint ignores timestamps/peer IDs	K2 (content-locality)	$\kappa(o)$ depends only on o's content fields
Metadata updates don't change fingerprint	K3 (non-key invariance)	Updating `description` after creation doesn't recompute κ
LWW per field (vv_dominates + timestamp)	ρ_k (total order)	argmax over (VV dominance, timestamp desc, agent_id asc)
`merge_entity_ops()`	CK-CRDT merge	Partitions by κ, selects representative via ρ_k
Phase 2: `entity_dedup_via_crdt()`	Representative selection	Groups by fingerprint, picks max(id) per group
Redirect map `{42: 99}`	Kernel of merge (Theorem 3)	Losers are redirected, not deleted; information loss = within-class loser set

B.2 Edge-Level Mapping

Paper 1 (Pipeline)	Paper 2 (Framework)	Mapping
`kg_edge_crdt` operations	Downstream CRDT	Edges reference entities; foreign-key dependency
`resolve_edge_endpoints()`	Theorem 2 (Layered No-Orphan)	Canonicalization at write time prevents orphans
`kg_entity_redirect` table	Durable redirect map	Write-time lookup ensures edges reference canonical IDs
Orphan guard (Phase 3)	Invariant 3 ([14] §5.2)	No edge references a non-canonical entity
`verify_crdt_consistency()`	Defense-in-depth	Post-hoc check (production has write-time prevention)

B.3 Convergence Claim Mapping

Paper 1 Claim	Paper 2 Theorem	How
Pipeline converges regardless of operation order	Theorem 4 (K1-K3 $\to$ convergence)	Fingerprint is deterministic (K1), content-local (K2), non-key invariant (K3)

Paper 1 Claim	Paper 2 Theorem	How
Concurrent add+remove: add wins	2P-Set (standard CRDT)	Not a CK-CRDT property; standard CRDT membership semantics
Different descriptions: entities coexist	Theorem 3 (kernel)	Different fingerprints → different key classes → no merge
Same descriptions: entities collapse	Theorem 1 (monotonicity)	Same fingerprint → same key class → max(id) wins
Redirect applied atomically to all edges	Theorem 2 (no-orphan)	Write-time canonicalization ensures invariant holds at write, not just after sync
Partial-replication convergence (`test_partial_replication_convergence`, `ck_crdt.py` property suite)	Theorem 4 (convergence, eventual broadcast fixed point)	Peers seeded with distinct op subsets project on partial bags, reconcile, and re-project to the same canonical state as the full-bag peer

B.4 What Paper 2 Adds Beyond Paper 1

Paper 1 proves convergence for one specific pipeline. Paper 2 proves convergence for the entire class:

1. **Generality:** Paper 1's results apply to `SHA-256(name, type, desc)`. Paper 2's results apply to any κ satisfying K1-K3. A different content key (e.g., `SHA-256(name, type)`) gets the same convergence guarantee for free.
2. **Necessity:** Both papers establish that (K1)–(K3) are necessary and sufficient for convergent deduplication — Paper 1 Theorem 3 (with counterexamples `TestK1Violation` / `TestK2Violation` / `TestK3Violation`) and Paper 2 Theorem 4 (the same counterexamples formalized in §6). (K1) and (K2) are necessary for convergence; (K3) is necessary for correct deduplication.
3. **Composition:** Paper 1 doesn't address how CK-CRDTs compose with other CRDTs. Paper 2 proves composition with delta-CRDTs (Theorem 7) and multi-key systems (Theorem 5).
4. **Limits:** Paper 1 doesn't identify where the pipeline breaks. Paper 2 identifies three structural failure modes (§8.4): key collisions, cross-class causality, and adaptive key cycles.
5. **Classification:** Paper 1 describes one system. Paper 2 classifies Docker, IPFS, Git, Nix, Yjs, Automerge, and Loro as instances or non-instances, explaining *why* each falls where it does.

References

[1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-Free Replicated Data Types," in *Stabilization, Safety, and Security of Distributed Systems*, vol. 7032 of *LNCS*, Springer, 2011, pp. 386–400.

[2] I. P. Fellegi and A. B. Sunter, "A Theory for Record Linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, Dec. 1969.

[3] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "A Comparison of String Distance Metrics for Name-Matching Tasks," in *Proceedings of IJCAI 2003*, 2003, pp. 73–77.

[4] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, "Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types," in *Proceedings of the 15th International Conference on Web Engineering (ICWE 2015)*, vol. 9114 of *LNCS*, Springer, 2015, pp. 675–678. doi: 10.1007/978-3-319-19890-3_55

[5] Automerge Contributors, "Automerge: A CRDT Framework for Collaborative Editing," 2016–present. [Online]. Available: https://github.com/automerge/automerge

[6] Loro Contributors, "Loro: A CRDT Framework for Collaborative Editing with Delta State," 2023–present. [Online]. Available: https://github.com/loro-dev/loro

[7] L. D. Ibáñez, H. S. Molli, P. Molli, and O. Corby, "Live Linked Data: Synchronising Semantic Stores with Commutative Replicated Data Types," *International Journal of Metadata, Semantics and Ontologies*, vol. 8, no. 2, art. 119, 2013.

[8] A.-C. N. Ngomo and S. Auer, "LIMES — A Time-Efficient Approach for Large-Scale Link Discovery on the Web of Data," in *Proceedings of IJCAI 2011*, 2011, pp. 2312–2317.

[9] J. Benet, "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[10] M. Zawirski, A. Bieniusa, V. Balegas, S. Duarte, C. Baquero, M. Shapiro, and N. Preguiça, "SwiftCloud: Fault-Tolerant Geo-Replication Integrated all the Way to the Client Machine," in *Proceedings of the 33rd IEEE International Symposium on Reliable Distributed Systems Workshops (SRDS-W 2014)*, 2014, pp. 30–33. doi: 10.1109/SRDSW.2014.33

[11] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[12] M. Buus, "Hypercore: An Append-only Log Built for Feeding Distributed Systems," 2018. [Online]. Available: https://hypercore-protocol.org/

[13] M. Kleppmann, "Making CRDTs Byzantine Fault Tolerant," in *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, 2022.

[14] S. Sadhu, "Conflict-Free Knowledge Graph Projection: A Three-Phase CRDT Pipeline for Multi-Agent Memory Systems," preprint, 2026.

[15] P. S. Almeida, A. Shoker, and C. Baquero, "Delta State Replicated Data Types," *Journal of Parallel and Distributed Computing*, vol. 111, pp. 162–173, 2018. doi: 10.1016/j.jpdc.2017.08.003

[16] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*, Springer, 2012.

[17] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[18] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone, "Logical Physical Clocks," in *Proceedings of the 18th International Conference on Principles of Distributed Systems (OPODIS 2014)*, vol. 8878 of *LNCS*, Springer, 2014, pp. 17–32. doi: 10.1007/978-3-319-14472-6_2

[19] G. Psaras et al., "Merkle-CRDTs: Merkle-DAGs meet CRDTs," 2020. [Online]. Available: https://research.protocol.ai/publications/merkle-crdts-merkle-dags-meet-crdts/

[20] P. Ramesh, V. Soundarapandian, and K. C. Sivaramakrishnan, "Sal: Multi-modal Verification of Replicated Data Types," arXiv:2603.27202, 2026.

[21] R. Mao et al., "Making CRDTs Not So Eventual," Proceedings of the VLDB Endowment, vol. 18, p. 349, 2025.